

ULA, Banco de Registradores, Unidade de Controle, Controle da ULA e Decodificador¹

André Felipe Wonsik Alves¹, Eber Felipe Barotti Louback², Erick Molina Gehring³

Universidade Tecnológica Federal do Paraná – UTFPR

*COCIC – Coordenação do Curso de Bacharelado em Ciência da Computação
Campo Mourão, Paraná, Brasil*

¹aalves.2020@alunos.utfpr.edu.br

²efblouback@gmail.com

³erickmolinagehring@alunos.utfpr.edu.br

¹ Trabalho desenvolvido para a disciplina de BCC2003 – Arquitetura e Organização de Computadores

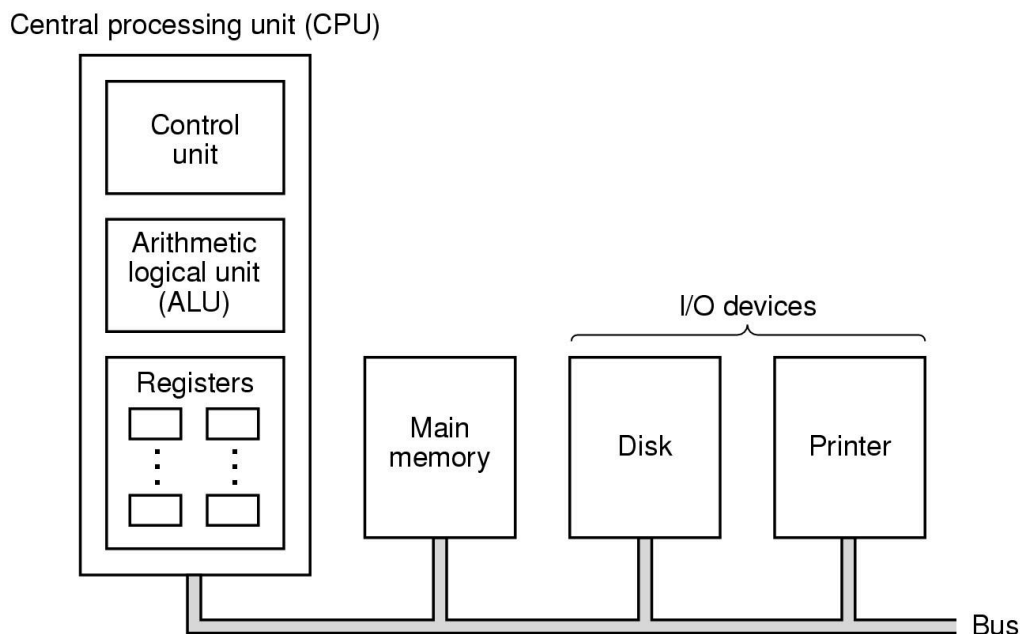
Resumo

O presente trabalho tem como principal objetivo a demonstração da construção de um caminho de dados e – consequentemente, de um processador. Durante o processo de criação, os seguintes componentes foram desenvolvidos: a Unidade Lógica Aritmética, o Banco de Registradores, o Decodificador de Instruções, a Unidade de Extensão de Sinal. Todo o circuito lógico foi construído dentro da aplicação *Logisim* e os componentes de soma, subtração, multiplexação e demultiplexação utilizados vêm pré-construídos no programa. Também vale ressaltar que a memória de dados e a memória de instruções foram disponibilizadas pelo professor em aula e, dessa maneira, utilizadas em nosso projeto.

1. Introdução

Para se criar um processador que segue uma arquitetura *MIPS*, deve-se analisar e projetar o chamado “caminho de dados”, representado na Figura 1, já que esse fluxo é o responsável por realizar as operações incumbidas, assim como alterar o estado da máquina. Essa ferramenta pode ser subdividida em quatro principais partes, sendo elas: Unidade Lógica Aritmética (*ULA*), Banco de Registradores, Decodificador e Unidade de Controle (*UC*).

Figura 1



Fonte:

<https://cecead.com/assuntos/disciplinas/organizacao-estruturada-de-computadores/aula-05-organizacao-estruturada-de-computadores/>

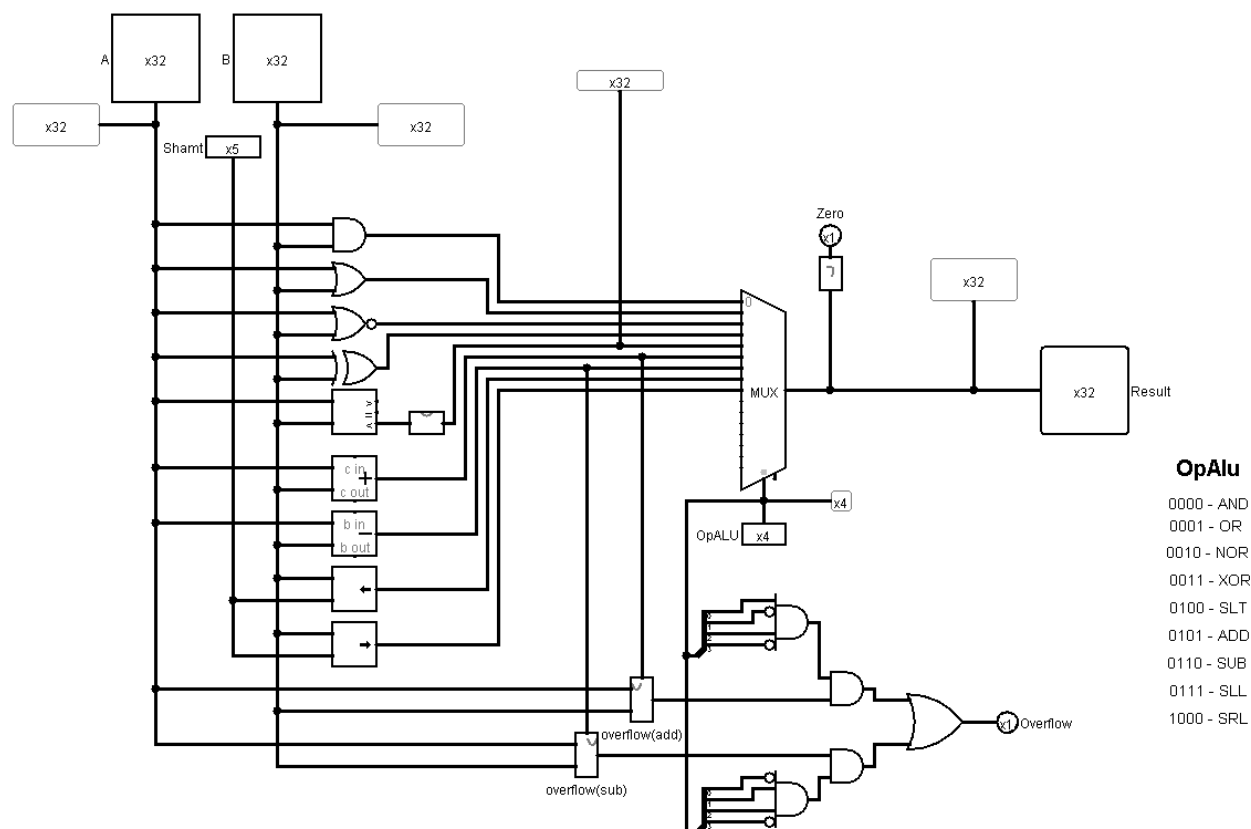
É fato que, para uma boa construção dessas ferramentas, deve-se antes entender seus conceitos e o funcionamento de cada uma dentro de um fluxo de trabalho no processador. Respectivamente, de acordo com Cardieri^[1] (2016) o primeiro elemento consiste em um

dispositivo que realiza operações lógicas aritméticas sobre números representados em circuitos lógicos; O segundo conceito indica em um componente digital com um conjunto de registradores que podem ser acessados de forma organizada, possibilitando operações de leitura e escrita de dados (Patterson e Hennessy, 2011)^[2].

2. Unidade Lógica Aritmética (ULA)

De acordo com o documento de especificação do trabalho, a *ULA* deve possuir entradas *A* e *B* com 32 bits de largura, uma *Operação ALU* com 4 bits, um pino *Shamt* com 5 bits, uma saída *Result* com 32 bits, um bit *Zero* e um bit *Overflow*. Para suas operações, já seguindo a ordenação na *ULA* a partir do primeiro bit, foram utilizadas as seguintes: *AND*, *OR*, *NOR*, *XOR*, *SLT*, *ADD*, *SUB*, *SLL* e *SRL*. Ressalta-se que os componentes de menor, soma, subtração, *MUX*, *shift-left-logical*, *shift-right-logical* utilizados foram os pré-construídos da aplicação *Logisim*. Para uma melhor compreensão, a Figura 2 indica como o circuito foi organizado.

Figura 2



Fonte: Autoria própria

No circuito, os componentes de *overflow* e para o bit *Zero* são de autoria própria de nosso grupo. Para sua análise detalhada, respectivamente, as sub-seções 2.1 e 2.2 apresentarão os

conceitos por trás dessas ferramentas. Sendo assim, o próximo tópico será a funcionalidade da *ULA*, exposto na seção 2.1

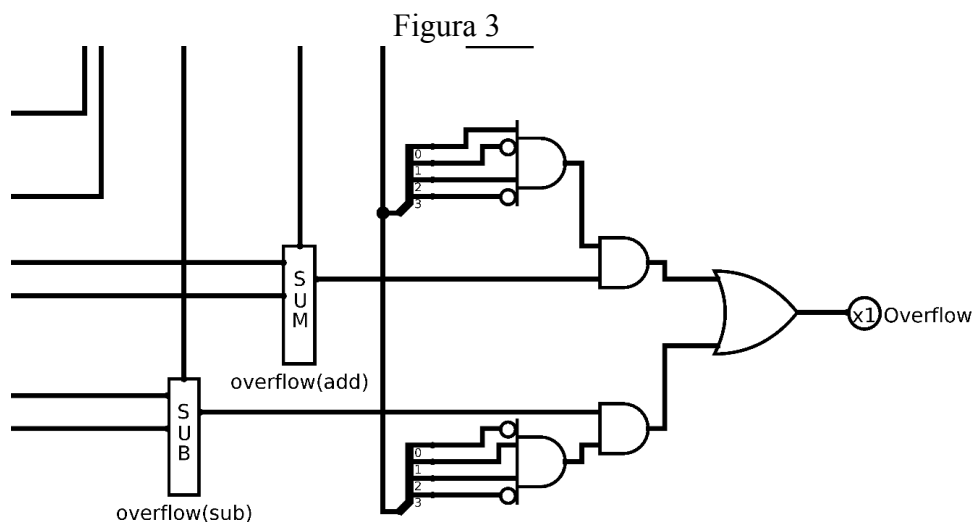
2.1 Funcionamento

A ULA desenvolvida recebe *A* e *B* como operandos – ambos com 32 bits –, além de um valor denominado *Shamt*, que indica a quantidade de casas a serem deslocadas em operações de deslocamento. No circuito em si temos operadores lógicos básicos como *AND*, *OR*, *XOR* e *NOR*. Ademais, nesse mesmo local é interessante ressaltar os componentes de soma e subtração, visto que há neles um circuito adicional para a detecção de *Overflow* (caso o resultado ultrapasse o intervalo de 32 bits), cuja lógica será apresentada no tópico 2.2.

Na figura 2, percebe-se que um componente “afunila” todas as outras operações, esse é o chamado Multiplexador (*MUX*), com a escolha das operações sendo feita por meio de *OpALU*. Na parte de saída temos o sinal *Zero*, cujo objetivo é indicar se todos os bits de *Result* são 0, o *Overflow* e o *Result* (indica o resultado final da operação escolhida pelo *MUX*).

2.2 Overflow

A saída *overflow* da ULA utiliza dois circuitos separados de overflow, um para a operação de adição e a outra para a de subtração. A saída só é 1 quando a saída do *overflow(add)* for 1 e a operação do *OpALU* for adição (operação de número 5) ou quando a saída do *overflow(sub)* for 1 e a operação do *OpALU* for subtração (operação de número 6).

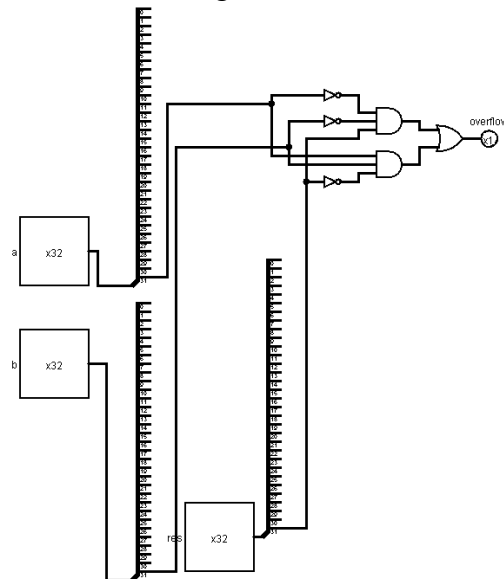


Fonte: Autoria própria.

No *overflow* para adição a saída só é 1 quando o bit mais significativo de *a* e *b* forem 0 (números positivos) e o resultado da adição (representado por *res*) tiver o bit mais

significativo 1 (número negativo) ou quando a e b forem 1 (números negativos) e o resultado da adição tiver o bit mais significativo 0 (número positivo). Como apresentado pela Figura 4.

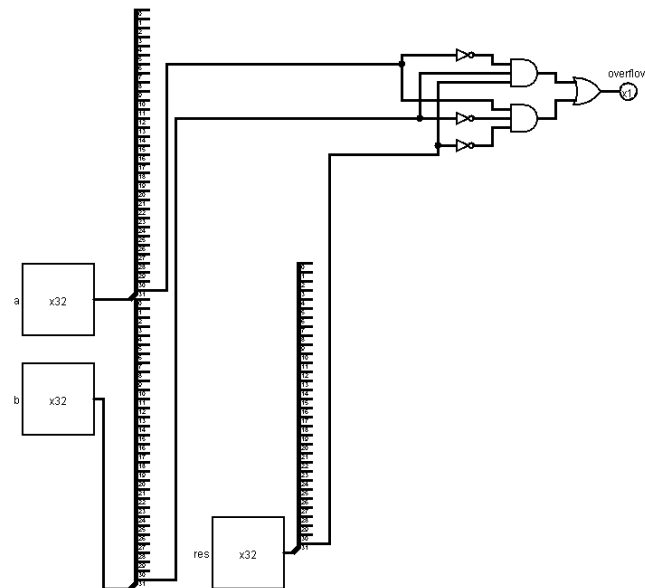
Figura 4



Fonte: Autoria própria.

No *overflow* para subtração a saída só é 1 quando o bit mais significativo de a for 0 (número positivo) e o de b for 1 (número negativo que se torna positivo na subtração) e o resultado da subtração (representado por res) ter o bit mais significativo 1 (número negativo) ou quando o bit mais significativo de a for 1 (número negativo) e o de b for 0 (número positivo que se torna negativo devido a subtração) e o resultado da subtração ter o bit mais significativo 0 (número positivo). Como apresentado pela Figura 5.

Figura 5

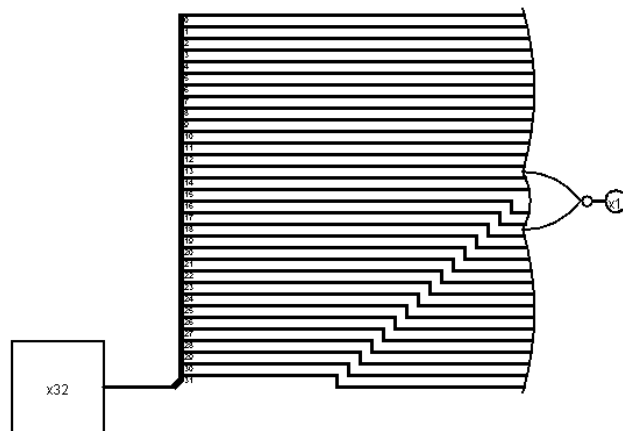


Fonte: Autoria própria.

2.3 Zero

A forma utilizada para descobrir se todos os bits de *result* são 0 utilizada foi por meio de uma porta *NOR* ligada a todos os bits do resultado.

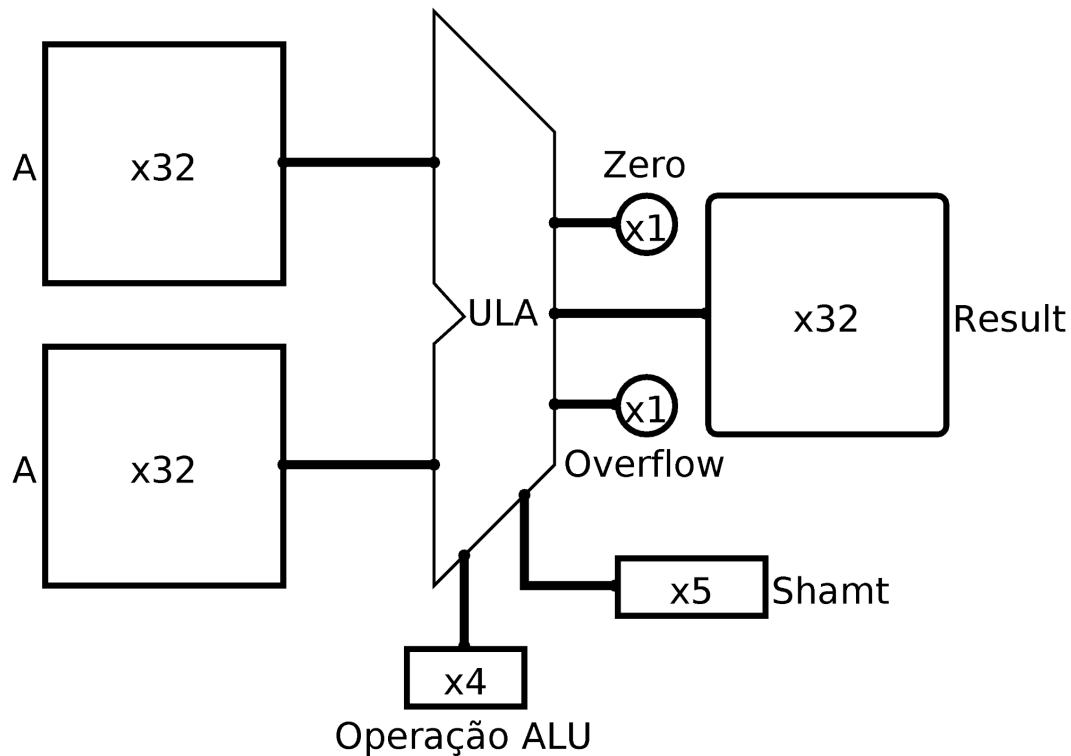
Figura 6



Fonte: Autoria própria.

Portanto, a Unidade Lógica Aritmética de nosso circuito é composta por todos os conceitos apresentados, como indica a Figura 7. Dessa maneira, a seção a seguir anuncia a especificação de nosso Banco de Registradores.

Figura 7



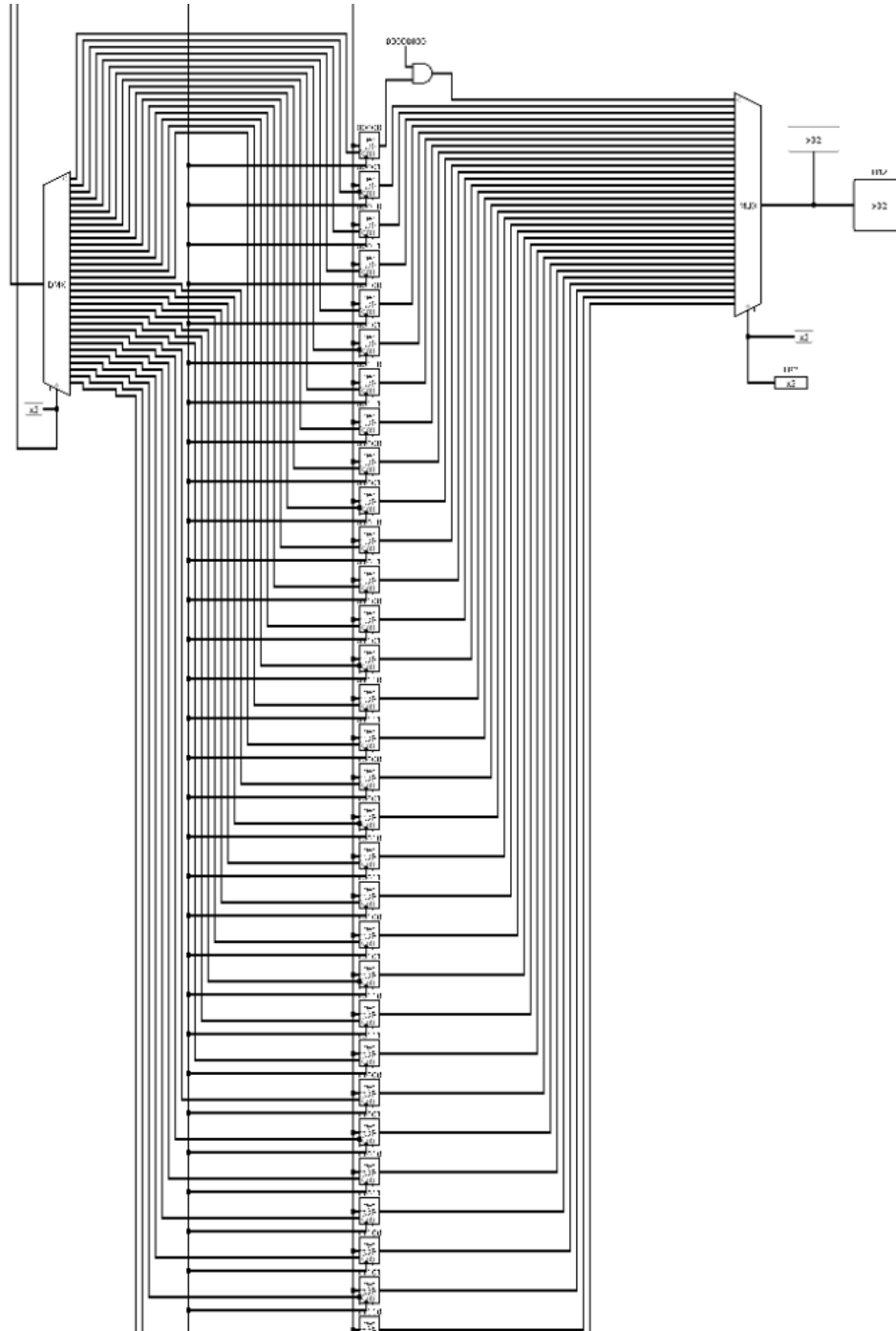
Fonte: Autoria própria.

3. Banco de registradores

Seguindo os pontos descritos no documento de especificação do trabalho, temos que: *LR1*, *LR2* e *WR* são identificadores de registradores com 5 bits, *WD* é uma entrada de 32 bits com o dado para escrever no registrador especificado em *WR*, *LD1* é o dado armazenado no registrador especificado em *LR1*, *LD2* é o dado armazenado no registrador especificado em *LR2*, *RegWrite* é um bit que habilita ou desabilita a escrita do dado contido em *WD* no registrador identificado em *WR* e, por fim, o registrador identificado por *00000* deve retornar sempre zero nos 32 bits e não deve permitir escrita. As Figuras 8 e 9 representam o circuito completo, com todos os elementos citados; foi necessária a separação da imagem em duas devido ao seu tamanho vertical preencher completamente mais de uma página.

Fonte: Autoria própria.

Figura 9: Segunda parte do Banco de Registradores



Fonte: Autoria própria.

3.1 Funcionamento

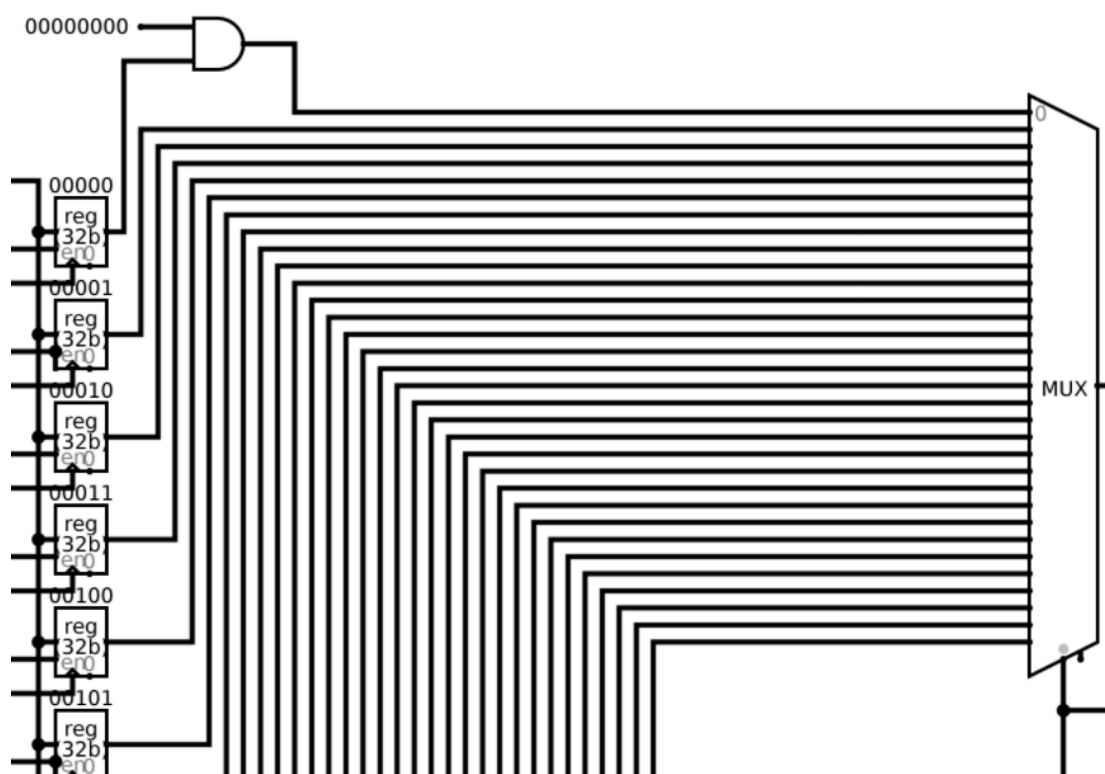
O banco de registradores desenvolvido recebe *WD* como uma entrada de 32 bits com o dado para escrever no registrador especificado em *WR*, sendo *WR* um identificador, de 5 bits, de registradores, que tem como função selecionar qual saída, no caso qual registrador, receberá o valor de dado por *WD*, isso ocorre graças a um Demultiplexador (*DEMUX*). Semelhante ao mesmo, ainda temos o *LR1* e o *LR2* que também são identificadores, de 5 bits, de registradores, porém esses selecionam qual saída se tornará entrada para *LD1* e *LD2* que são os dados armazenados no registrador especificado em *LR1* e *LR2*, isso a partir de um Multiplexador (*MUX*).

Há também a presença de um *RegWrite* de 1 bit que habilita ou desabilita a escrita do dado contido em *WD* no registrador identificado em *WR*. Importante ressaltar que o registrador identificado por *00000* retorna sempre zero nos 32 bits e não permite escrita, cuja implementação será apresentada no tópico 3.2.

3.2 Registrador 00000

Em nosso projeto, optamos por criar o registrador *00000* por meio da porta lógica AND, como demonstrado na figura abaixo.

Figura 10

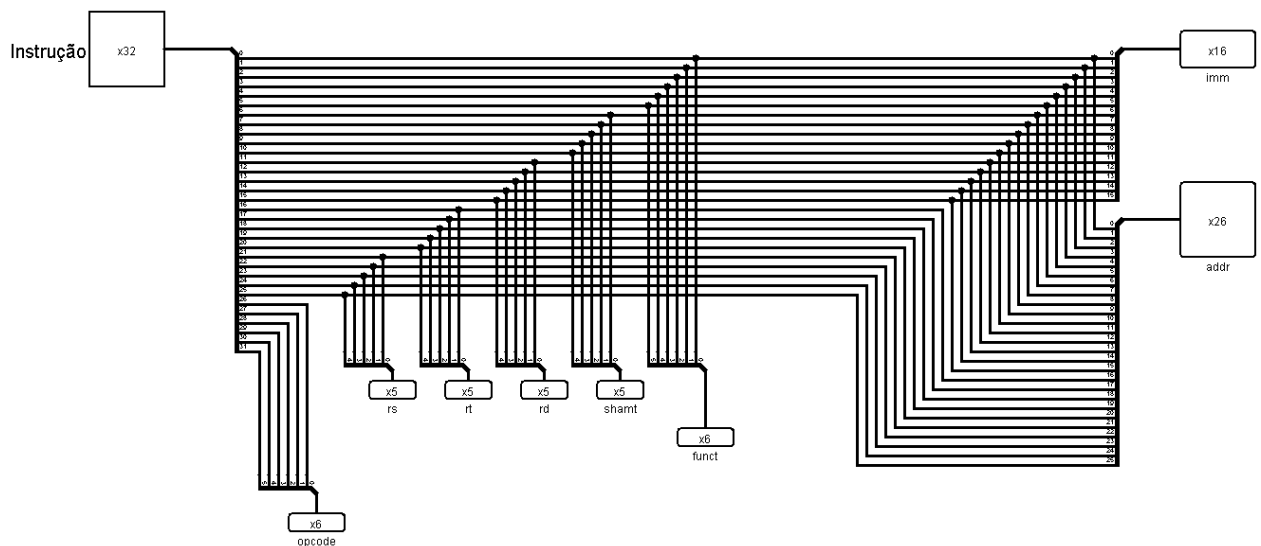


Fonte: Autoria própria.

O registrador *00000* é amplamente utilizado nas operações simples de um código, como por exemplo em *beq*, *bne*, *add*, *sub*, entre outros.

4. Decodificador de Instruções

O Decodificador de Instruções recebe uma instrução, de 32 bits de tamanho, dividindo-a entre os seguintes campos: opcode, rs, rt, rd, shamt, funct, imm e addr. Apesar de definir todos esses campos, não é trabalho do Decodificador decidir qual deve ser utilizado, apenas disponibilizando todos eles independentemente do tipo de instrução fornecida.

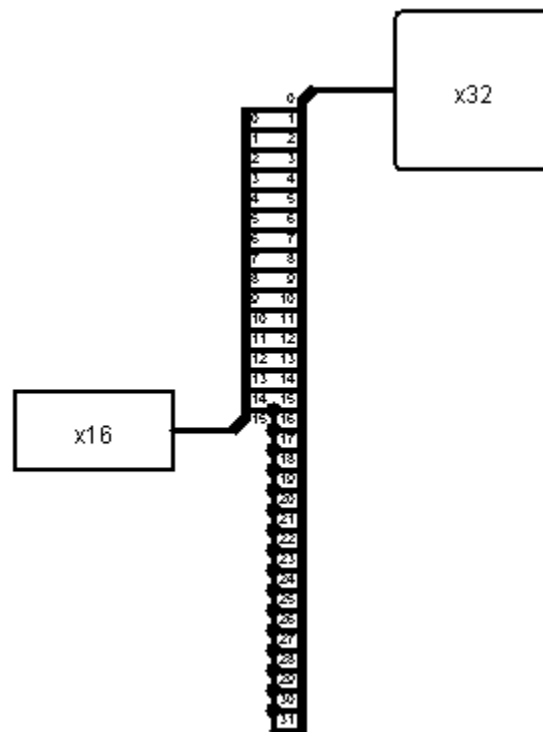


Fonte: Autoria própria.

5. Unidade de Extensão de Sinal

A Unidade de Extensão de Sinal, simplesmente, converte um valor imediato com 16 bits para outro com 32 bits. Esse processo deve acontecer para que o número possa ser utilizado nas operações localizadas na ULA. O circuito consiste no uso de distribuidores logicamente organizados para que o bit mais significativo da entrada seja repetido para todos os bits restantes da saída, mantendo seu valor e sinal inicial. A figura 11 apresenta o circuito ensinado.

Figura 11



Fonte: Autoria própria.

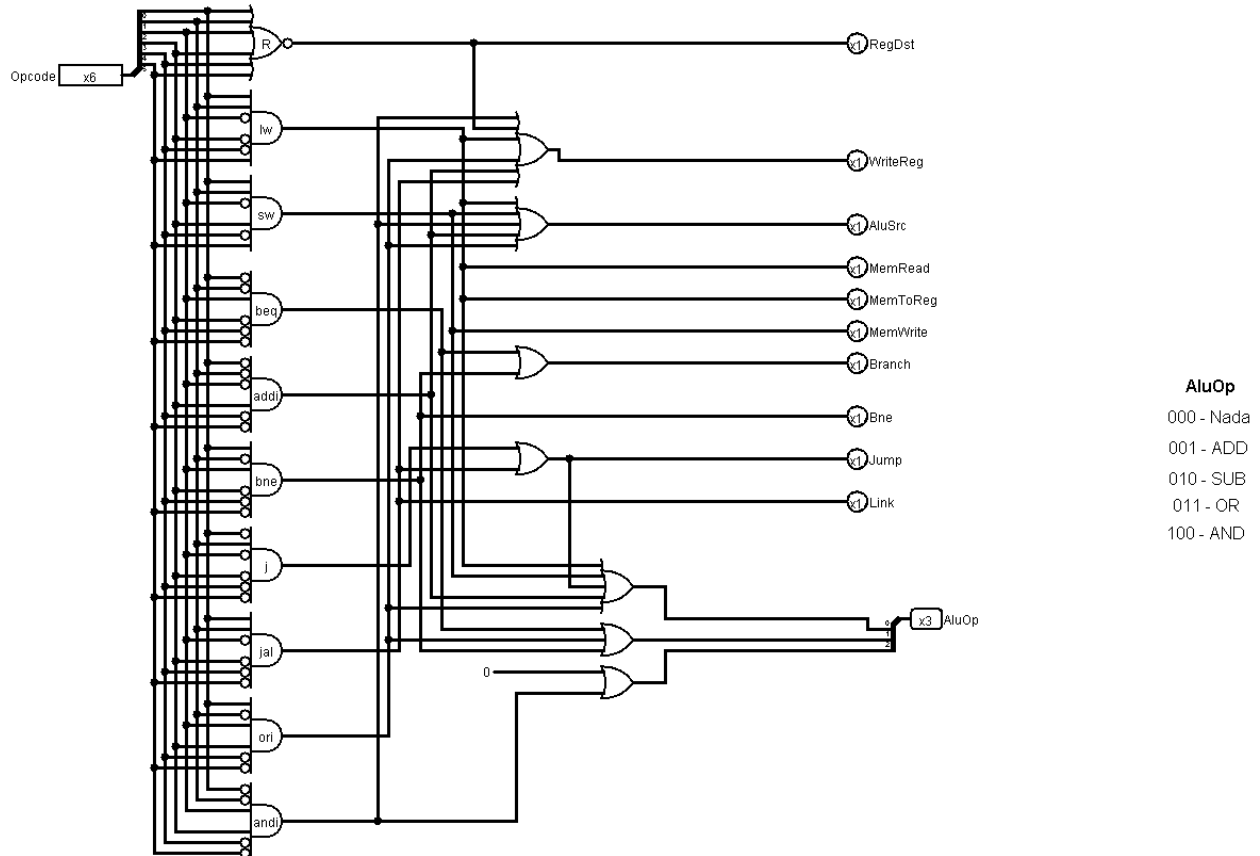
6. Unidade de Controle Principal

6.1. Funcionamento

A Unidade de Controle Principal (UCP) tem como responsabilidade definir os sinais de controle do circuito, baseando-se no *Opcode* proveniente do Decodificador de Instruções. Os sinais de controle são:

- **RegDst**: É ativado somente quando a operação é do tipo R (*Opcode* é 000000);
- **WriteReg**: Ativo quando a operação é do tipo R ou é *lw*, *addi*, ou *jal*;
- **AluSrc**: Ativo somente quando a operação é de tipo I, neste caso abrangendo somente as operações *addi*, *lw* e *sw*;
- **MemRead**: Ativo somente quando a operação é *lw*;
- **MemToReg**: Também só se encontra ativo quando a operação é *lw*;
- **MemWrite**: Ativo somente em *sw*;
- **Branch**: Ativo quando a operação é *beq* ou *bne*;
- **Bne**: Somente ativo quando a operação é *bne*;
- **Jump**: Ativo quando a operação é *j* ou *jal*;
- **Link**: Só está ativo quando a operação é *jal*;
- **AluOp**: Pode ser encontrada nos estados 000, 001, 010, 011 ou 100. Se a operação é do tipo R *AluOp* tem o valor de 000, com a operação da ULA sendo controlada pelo *func*. Quando a operação é *addi*, *lw* ou *sw* tem o valor de 001, chamando a operação de adição na ULA. Quando a operação é *beq* ou *bne* tem o valor de 010,

realizando a operação de subtração na ULA. Quando a operação é *ori* tem o valor de 011, efetuando a operação de or na ULA. E por último, quando a operação é *andi* tem o valor de 100, executando a operação and na ULA.



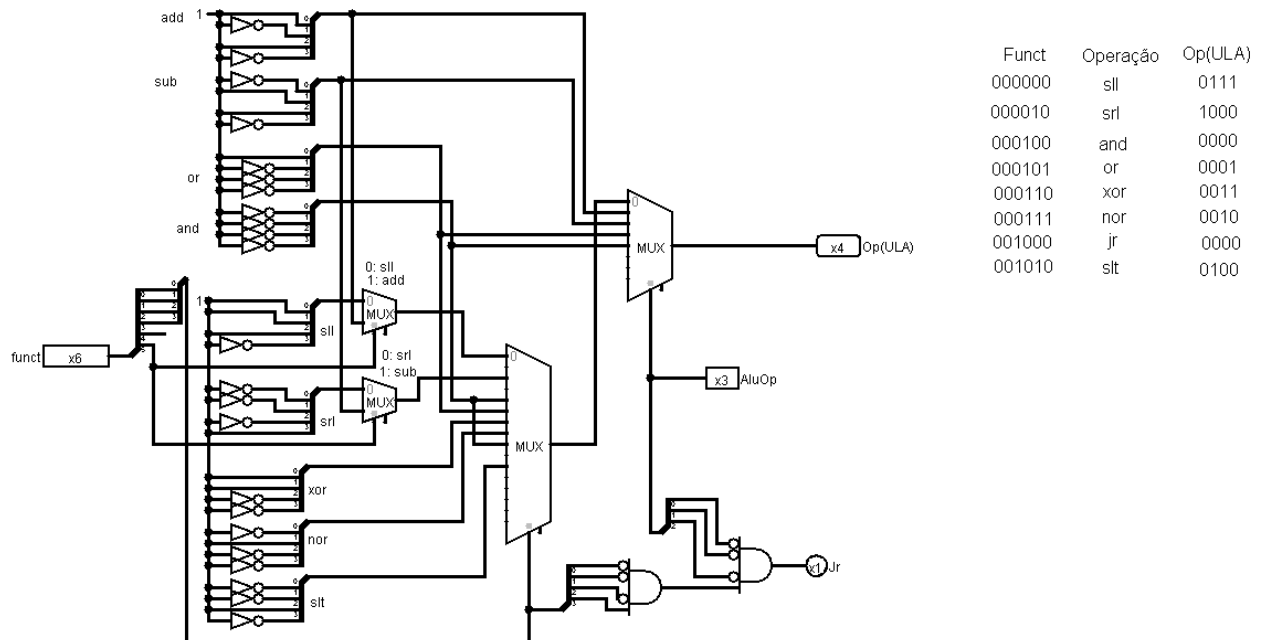
Fonte: Autoria própria.

7. Unidade de Controle da ULA

A Unidade de Controle da ULA utiliza o *funct* vindo do Decodificador de Instruções e a *AluOp* vinda da Unidade de Controle Principal. A *AluOp* é conectada a um multiplexador para poder selecionar o endereço da operação desejada dentro da ULA. Para utilizar o *funct* para selecionar a operação tipo R foi utilizado um MUX de quatro bits que utiliza os seus 4 bits menos significativos já que eles se repetem apenas duas vezes nas operações disponíveis na nossa ULA, estes sendo *add/sll* e *sub/srl*.

Para decidir qual das operações com os mesmos bits menos significativos deve ser selecionada foi utilizado um MUX de 1 bit para cada par de operações. Ele utiliza o bit mais significativo do *funct* (que é a única diferença entre as operações) para escolher qual dos dois deve ser selecionado. Por exemplo, se o *funct* é 100000 a operação a *Op(ULA)* será 1001, que significa adição na nossa ULA, mas se o *funct* for 000000 a *Op(ULA)* será 1011, que é a operação *sll*.

Caso a operação selecionada seja *jr* o sinal *Jr* é ativado, caso o *AluOp* seja 000 para evitar uma ativação errônea, e envia o sinal 0000 para *Op(ULA)*, o sinal de *and*, apenas para evitar que ela seja XXXX e que isso cause algum erro na ULA.



Fonte: Autoria pr

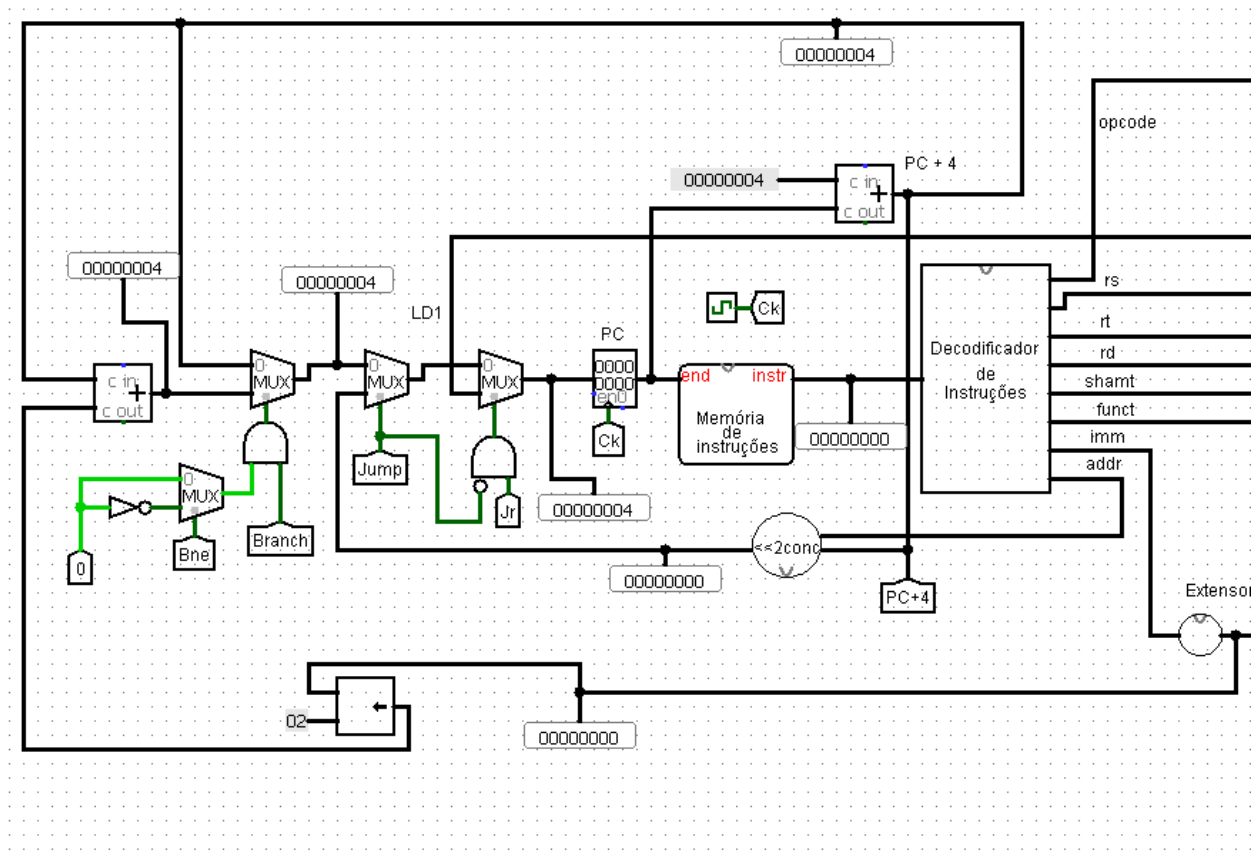
8. Caminho de dados monocicloópria.

8.1. Especificação

A partir do último tópico, ainda havia um número significativo de componentes necessários para o funcionamento completo do processador que necessitam ser abordados. Em específico, eles são: a estrutura que lida com as operações de desvio, o *Program Counter*, a Memória de Instruções, a Memória de Dados e o componente <<2conc.

Eles serão abordados em detalhe nos próximos sub-tópicos.

8.2. Operações de Desvio, PC e Memória de Instruções

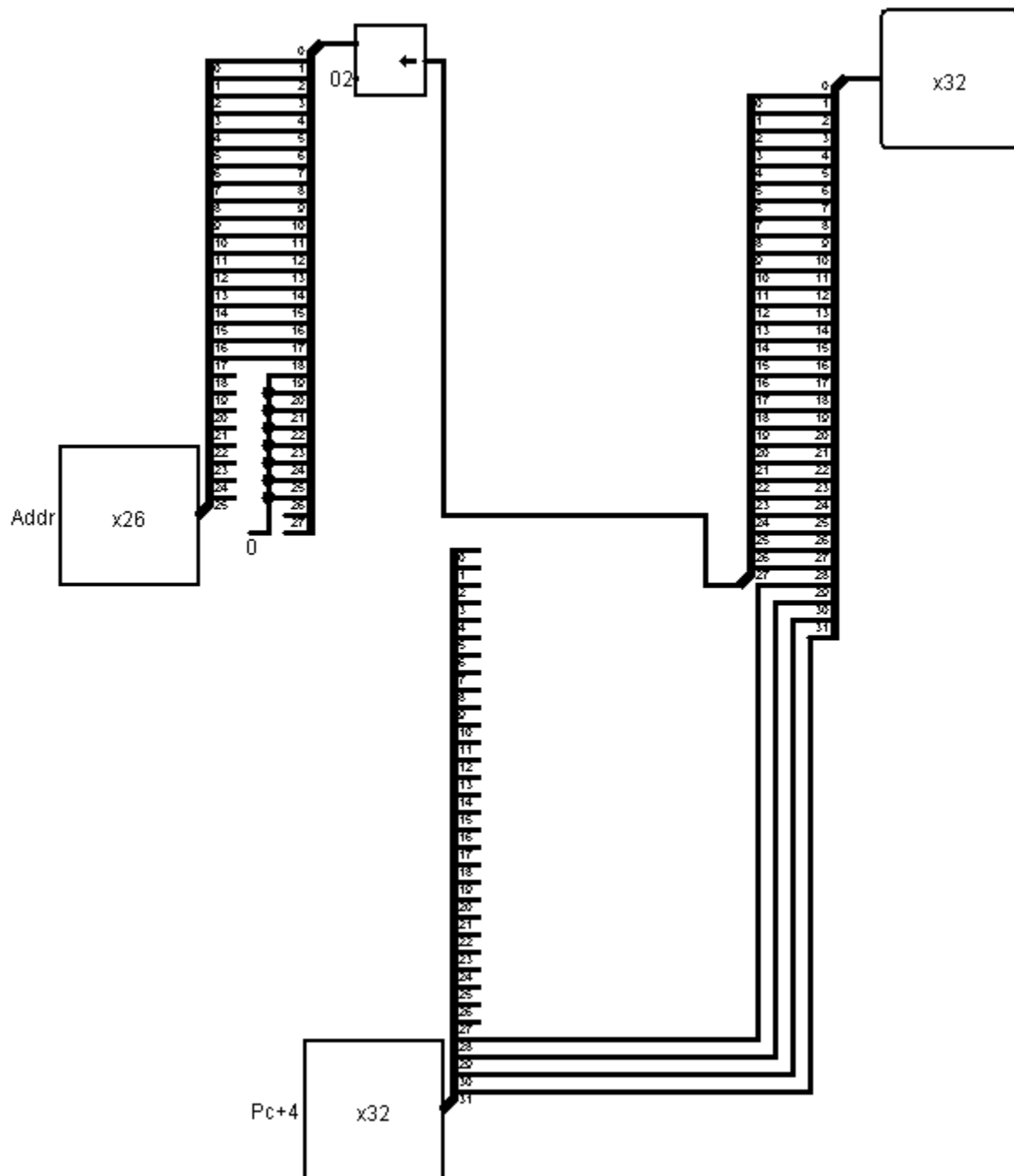


Fonte: Autoria própria.

O PC é um registrador que serve para armazenar o endereço da instrução que deve ser processada. O seu valor é, geralmente, o valor já armazenado no componente somado com 4, que passa a memória de instruções para a próxima posição. No entanto, seu valor pode ser alterado para algo além disso quando há uma instrução de desvio.

Quando uma instrução *beq*, *bne*, *j*, *jal* ou *jr* é processada, elas podem mandar os sinais *Bne*, *Branch*, *Jump* e *Jr*.

Quando a instrução é *bne* os sinais de controle *Bne* e *Branch* são ativados, usando um *Mux* conectado ao sinal 0, utilizado para determinar se o número é ou não igual e caso seja, o endereço enviado é $PC + 4 + (imm \ll 2)$, que é PC+4 somado com o valor imediato deslocado 2 casas para a esquerda, com *beq* fazendo essencialmente a mesma coisa.

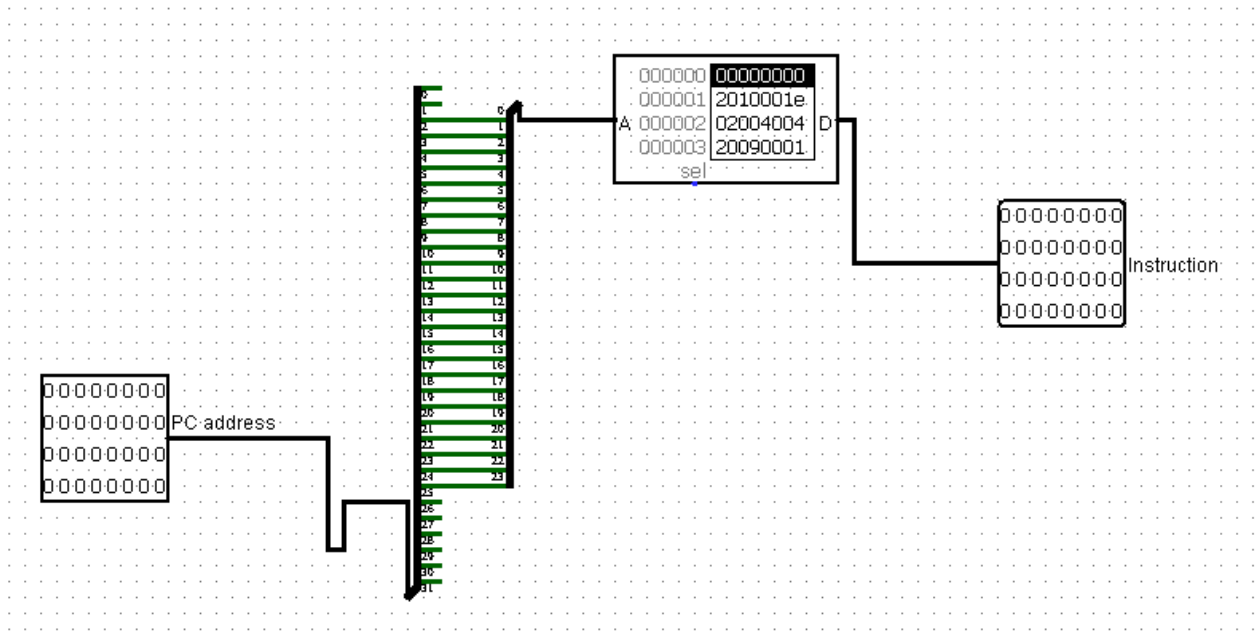


Fonte: Autoria própria.

O componente <<2conc é utilizado pela instrução de *j* ao emitir o sinal *jump*. O componente consiste em pegar o valor de PC+4, deslocá-lo 2 casas para a esquerda e zerar os 8 bits mais significantes já que não há sistema operacional para ser considerado. Após isso, os 4 bits mais significativos de PC+4 são concatenados ao resultado da última operação e isso é então mandado para o PC.

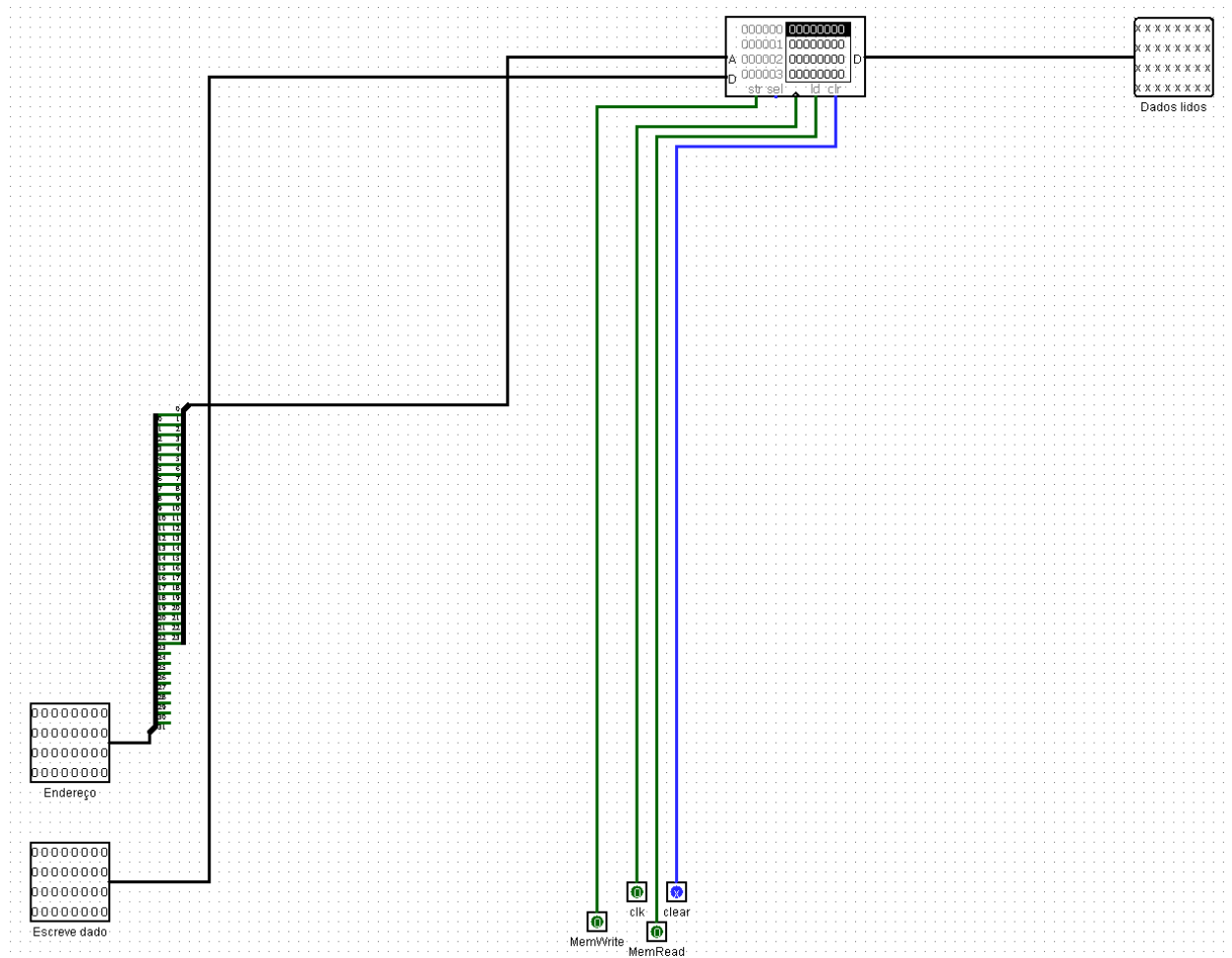
O *jr* é ativo somente quando a operação é *Jump Register*, ela seleciona o valor vindo do registrador *\$ra* e somente seleciona quando o sinal *jr* está ativo e o *jump* não está, com o objetivo de evitar que o *jr* seja ativado indevidamente.

A Memória de Instruções recebe um endereço 32 bits e utiliza 24 desses bits para determinar a posição da instrução e retorná-la.



Fonte: Componente providenciado no Moodle pelo Prof.Dr.João Fabrício Filho

A Memória de Dados, essencialmente recebe apenas um endereço que leva para o lugar na memória que os dados devem ser escritos ou lidos, ação definida pelos sinais MemWrite ou MemRead.



9. Execução

9.1. Código em Linguagem de Alto Nível (C)

```
#include <stdio.h>

// Função Normal (Chamada de função folha)
int soma(int a, int b) {
    return a + b;    //Add
}

// Função Não-Folha (Chama outra função)
int multiplicarPor2(int a, int b) {
    return soma(a, b) * 2; //Sll por 2
}

int main() {
    // Variáveis para os testes
    int x = 3, y = 2;    //Addi // $s1 e $s2
    int resultado;        //Add ou Addi // $s0

    // Instruções R (Add, Sub, And, Or, Xor)
    resultado = x + y;    // Add
    resultado = x - y;    // Sub
    resultado = x & y;    // And
    resultado = x | y;    // Or
    resultado = x ^ y;    // Xor

    // Instruções com Imediato (Addi, Andi, Ori)
    resultado = x + 5;    // Addi
    resultado = x & 5;    // Andi
    resultado = x | 5;    // Ori
```

```
// Desvios Condicionais (Beq, Bne)
if (x == y) {
    resultado += y;
} else {
    resultado += x;
}

if (x > y) { // Slt
    resultado = x+y;
}

// Operações de Memória (Lw, Sw)
int memoria[10]; //No assembly utilizamos a pilha!
memoria[0] = 42; // Sw
resultado = memoria[0]; // Lw

// Laços de Repetição não Aninhados (For, While)
//For
for (int i = 0; i < x; i++) {
    resultado+=i;
}

//While
int i = 0;
while (i < y) {
    resultado+=i;
    i++;
}

// Laço de Repetição Aninhado (For dentro de For)
for (int i = 0; i < x; i++) {
    for (int j = 0; j < y; j++) {
        resultado+=i+j;
    }
}
```

```
// Chamada de Função Folha
resultado = soma(x, y);

// Chamada de Função Não-Folha
resultado = multiplicarPor2(x, y);

return 0;
}
```

9.2. Tradução em Assembly

```
Main:
# Inicializa variáveis x, y
addi $s1, $0, 3          # x = 3
addi $s2, $0, 2          # y = 2
# Variável resultado
add $s0, $0, $0          # resultado = 0

# Instruções R (Add, Sub, And, Or, Xor)
add $s0, $s1, $s2        # Add: resultado = x + y
sub $s0, $s1, $s2        # Sub: resultado = x - y
and $s0, $s1, $s2        # And: resultado = x & y
or  $s0, $s1, $s2        # Or: resultado = x | y
xor $s0, $s1, $s2        # Xor: resultado = x ^ y

# Instruções com Imediato (Addi, Andi, Ori)
addi $s0, $s1, 5          # Addi: resultado = x + 5
andi $s0, $s1, 5          # Andi: resultado = x & 5
ori  $s0, $s1, 5          # Ori: resultado = x | 5
```

```
# Desvios Condicionais (Beq, Bne)
bne $s1, $s2, Else1 # se x != y, vai para o Else
add $s0, $s0, $s2 # se x == y, continua aqui -> resultado += y
j MaiorQue

Else1: add $s0, $s0, $s1 #resultado do Else -> resultado += x

MaiorQue:
    slt $t1, $s1, $s2
    beq $t1, $0, Memoria
    add $s0, $s1, $s2 #resultado = x+y

Memoria:
    addi $sp, $sp, -4 # Reserva espaço na pilha
    addi $t2, $0, 42 # Define o valor 42
    sw $t2, 0($sp) # Armazena 42 na pilha
    lw $s0, 0($sp) # Carrega o valor salvo para $s0
    addi $sp, $sp, 4 # Libera o espaço da pilha

    add $t3, $0, $0 # i do For

For:
    slt $t4, $t3, $s1
    beq $t4, $0, Inicia_While
    add $s0, $s0, $t3
    addi $t3, $t3, 1
    j For

Inicia_While:
    add $t3, $0, $0 # i do While
```

```
While:
    slt $t4, $t3, $s2
    beq $t4, $0, Inicia_For_Aninhado
    add $s0, $s0, $t3
    addi $t3, $t3, 1
    j While

Inicia_For_Aninhado:
    add $t3, $0, $0 # i do While

For_Aninhado:
    slt $t4, $t3, $s1
    beq $t4, $0, Chamada_Funcao
    add $t5, $0, $0 # j do For Aninhado

For_Aninhado2:
    slt $t6, $t5, $s2
    beq $t6, $0, Fim_For_Aninhado
    add $t7, $t3, $t5
    add $s0, $s0, $t7
    addi $t5, $t5, 1
    j Fim_For_Aninhado

Fim_For_Aninhado:
    addi $t3, $t3, 1
    j For_Aninhado

Chamada_Funcao:
    # Chamada de Função Folha
    #resultado = soma(x, y);
    add $a0, $0, $s1
    add $a1, $0, $s2
    jal Soma
    add $s0, $0, $v0
```

```
# Chamada de Função Não-Folha
#resultado = multiplicarPor2(x, y);
jal MultiplicarPor2
add $s0, $0, $v0
j Fim

# Função Soma (Chamada de função folha)
Soma:
    add $v0, $a0, $a1      # soma a0 e a1, e armazena em v0 (retorno)
    jr $ra                # retorna para a chamada anterior

# Função MultiplicarPor2 (Chama soma)
MultiplicarPor2:
    # Chama Soma
    addi $sp, $sp, -4      # Salvando na pilha
    sw $ra, 0($sp)
    jal Soma
    lw $ra, 0($sp)
    # multiplica o resultado por 2
    sll $v0, $v0, 1        # desloca v0 para a esquerda (multiplica por 2)
    addi $sp, $sp, 4
    jr $ra                # retorna para a chamada anterior

Fim:
    j Fim
```

9.3. Correspondente na Linguagem de Máquina (Hexadecimal)

```
20110003
20120002
00008020
```


02328020
02328022
02328024
02328025
02328026
22300005
32300005
36300005
16320002
02128020
0810000f
02118020
0232482a
11200001
02328020
23bdfffc
200a002a
afaa0000
8fb00000
23bd0004
00005820
0171602a
11800003
020b8020
216b0001
08100018
00005820
0172602a
11800003
020b8020
216b0001
0810001e
00005820
0171602a
11800009
00006820
01b2702a

```
11c00004
016d7820
020f8020
21ad0001
0810002d
216b0001
08100024
00112020
00122820
0c100036
00028020
0c100038
00028020
0810003f
00851020
03e00008
23bdfffc
afbf0000
0c100036
8fbf0000
00021040
23bd0004
03e00008
0810003f
```


11. Referências

- [1] CARDIERI, Paulo: Unidade Lógica e Aritmética. Acessado em 2024 pelo link: www.decom.fee.unicamp.br/~cardieri/MaterialEA773/EA773_2S2016_Roteiro3.pdf. Material de aula, Campinas, São Paulo, 2016.
- [2] PATTERSON, David A. Hennessy, John L. Organização e Projeto de Computadores. Disponível em: Minha Biblioteca, (5a. edição). Grupo GEN, 2017